Vietnam National University, Ho Chi Minh City
University of Technology
Faculty of Computer Science and Engineering

DISCRETE STRUCTURE
(CO1007)

Assignment

A ALGORITHMS*

(UPDATE October 28th, 2025)

Instructor(s): Nguyễn An Khương, *CSE-HCMUT*
Trần Tuấn Anh,*CSE-HCMUT*
Mai Xuân Toàn, *CSE-HCMUT*
Trần Hồng Tài, *CSE-HCMUT*

Note: **To reduce the chance and effectiveness of cheating cases, there will be harmony questions in the final exam.**

Ho Chi Minh City, July 2025

Table of Contents

1 Introduction

1.1 Graph Theory

Graph theory is a foundational area of discrete mathematics that studies the relationships between objects using vertices and edges. A graph $G = (V, E)$ consists of a set of vertices V and a set of edges E, where each edge connects a pair of vertices. Graphs can be directed or undirected, weighted or unweighted, and may represent a wide range of systems such as networks, maps, or decision trees. In computer science, graphs are used to model data structures, optimize routes, and analyze connectivity. Directed graphs (digraphs) are particularly useful in pathfinding problems, where each edge has a direction and possibly a cost associated with traversal. Weighted graphs assign numerical values to edges, representing distances, time, or other metrics. Graph theory provides the mathematical framework for designing and analyzing algorithms that operate on these structures, including traversal, search, and optimization techniques. Understanding graph properties—such as connectivity, cycles, and shortest paths—is essential for solving complex problems in fields like artificial intelligence, logistics, and operations research. In this project, we use a directed weighted graph to represent a maze, where each cell is a node and each valid move is an edge.

1.2 Path in a Graph

A path in a graph is a sequence of vertices connected by edges, representing a route from one point to another. Formally, a path from vertex v_0 to v_n is a sequence $v_0, v_1, \ldots, v_n$ such that each pair (v_i, v_{i+1}) is an edge in the graph. The length of a path is typically defined as the number of edges it contains, while the cost of a path in a weighted graph is the sum of the weights of its edges. Paths can be simple (no repeated vertices), cyclic (starting and ending at the same vertex), or optimal (minimizing cost or length). In practical applications, paths are used to model navigation, communication, and decision-making processes. For example, in a maze represented as a graph, a path corresponds to a sequence of moves from the entrance to the exit. Identifying valid paths requires checking adjacency and avoiding obstacles or invalid transitions. In optimization problems, the goal is often to find the shortest or least-cost path among many possibilities. Algorithms like Dijkstra's and A* are designed to efficiently explore graphs and identify optimal paths based on specific criteria.

1.3 Pathfinding Algorithms

Pathfinding algorithms are computational techniques used to determine a viable or optimal route between two points in a graph. These algorithms play a crucial role in various domains such as robotics, logistics, computer games, and navigation systems. At their core, pathfinding methods explore the structure of a graph to identify a sequence of edges that connect a start node to a goal node, often while minimizing cost, distance, or time.

In weighted graphs, each edge carries a numerical value representing the cost of traversal. Pathfinding algorithms must account for these weights to avoid inefficient or costly routes. Some algorithms guarantee the shortest path, while others prioritize speed or simplicity, depending on the application. Common strategies include breadth-first search (BFS), depth-first search (DFS), and uniform cost search, each with its own strengths and limitations.

Effective pathfinding depends on how the graph is represented, how neighbors are evaluated, and how decisions are made during traversal. In maze-solving problems, for example, the graph is typically a grid, and the algorithm must navigate around walls, dead ends, and open paths.

The final output is a sequence of nodes forming a valid path from the start to the goal, often reconstructed by tracing back through visited nodes once the goal is reached.

1.4 The Shortest Path Problem

The shortest path problem is a fundamental question in graph theory and computer science, concerned with finding the minimum-cost path between two vertices in a graph. Given a weighted graph $G = (V, E)$, where each edge $(u, v) \in E$ has an associated non-negative weight $w(u, v)$, the goal is to determine a path from a source vertex s to a destination vertex t such that the sum of the edge weights along the path is minimized.

This problem has numerous real-world applications, including GPS navigation, network routing, transportation planning, and robotics. Depending on the graph's structure and constraints, different algorithms may be employed. For instance, Dijkstra's algorithm efficiently solves the single-source shortest path problem in graphs with non-negative weights, while the Bellman-Ford algorithm can handle graphs with negative edge weights. In cases where all edge weights are equal, a breadth-first search (BFS) can be used to find the shortest path in terms of the number of edges.

The shortest path problem also serves as the foundation for more advanced optimization techniques, such as those used in heuristic search, dynamic programming, and reinforcement learning. Understanding this problem is essential for designing intelligent systems that operate efficiently in complex environments.

1.5 A* Algorithm

The A* (A-star) algorithm is a widely used pathfinding and graph traversal technique in computer science and artificial intelligence. It is designed to find the shortest path from a starting node to a goal node in a weighted graph or grid-based environment.

A* combines the strengths of Dijkstra's algorithm and Greedy Best-First Search by using a cost function defined as:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ is the actual cost from the start node to the current node.
- $h(n)$ is the heuristic estimate of the cost from the current node to the goal.
- $f(n)$ is the total estimated cost of the cheapest solution through node n.

If the heuristic $h(n)$ is admissible (i.e., it never overestimates the true cost to reach the goal), A* is guaranteed to find the optimal path. The choice of a heuristic has a significant impact on the algorithm's efficiency and accuracy.

A* is commonly applied in:

- Maze and map navigation
- Puzzle solving (e.g., 8-puzzle)
- Robotics and autonomous movement
- Game AI and route planning

For example, the graph below illustrates a simple example of the A* pathfinding algorithm. Each node is labeled with its name, heuristic value $h(n)$, and total cost estimate $f(n) = g(n) + h(n)$. In this graph:

- Node **S** is the **start node**.
- Node **G** is the **goal node**.
- The optimal path is highlighted in red: **S** $\rightarrow$ **A** $\rightarrow$ **D** $\rightarrow$ **G**.

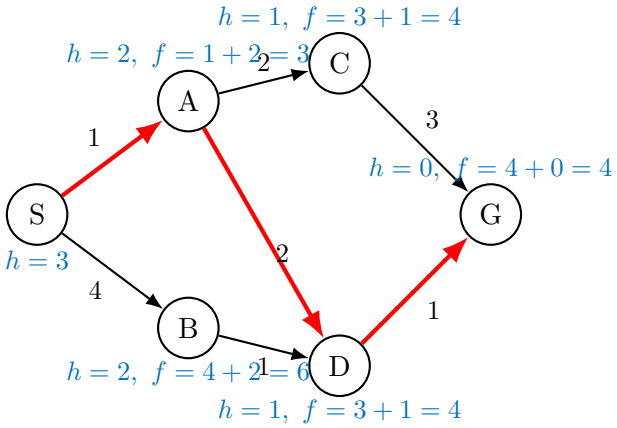

Fig. 1: *Example for A* algorithm*

In this project, students will implement A* in various contexts and explore its behavior under different heuristic strategies.

2 Project Requirements

2.1 Detailed Programming Tasks

Students are required to complete the 4 programming tasks described in this section, each involving the application of the A* search algorithm to solve a specific problem. All implementations must be written in **C++**, using only the default libraries provided by the LMS CodeRunner environment, which include: `<iostream>`, `<fstream>`, `<string>`, `<cmath>`, `<vector>`, and `<algorithm>`. Finally, students must submit the following:

- `PathNode.h` and `PathNode.cpp` for the `PathNode` struct and related functions.
- `Algo.h` and `Algo.cpp` containing all functions that solve the tasks.
- A `main.cpp` file that demonstrates the program's functionality using sample input and output.
- A report file that answers the report requirements for each task, named after your student ID (e.g., `24252628.pdf`).

1. **Shortest Path Using A* with Adjacency Matrix (2.5 pts):** Given a graph with n vertices represented by an adjacency matrix, implement the A* algorithm to find the shortest path from a start vertex to a goal vertex.
   - **Input:** Adjacency matrix, start vertex, goal vertex.

- **Output:** A linked list containing the path from start to goal, along with the values of $f(n)$, $g(n)$, and $h(n)$ at each vertex. Each node in the linked list should be named using the vertex index, starting from 0.
- **Heuristic Description:** The heuristic function $h(n)$ should estimate the cost from the current vertex to the goal based on the number of vertices remaining to be visited. The cost-so-far function $g(n)$ represents the number of vertices traversed from the start to the current vertex. The total cost function is defined as:

$$f(n) = g(n) + h(n)$$

  where:
  - $g(n)$: Number of vertices traversed from the start to vertex n.
  - $h(n)$: Estimated minimum number of vertices remaining from vertex n to the goal.
- **Report Requirement:** Students must describe how $h(n)$ is calculated and provide a manual example run for this task.

2. **Shortest Path in 2D Space with Two Heuristics (2.5 pts):** Given a graph represented by a weight matrix, where each vertex has coordinates (x, y) in 2D space, implement the A* algorithm to find the shortest path between two vertices using two heuristic modes.

   - **Input:** Adjacency matrix (with non-negative weights), list of vertex coordinates, start vertex, goal vertex, heuristic mode.
   - **Output:** A linked list containing the path from start to goal, along with the values of $f(n)$, $g(n)$, and $h(n)$ at each vertex. Each node in the linked list should be named using its 2D coordinate in the form (x, y).
   - **Cost Function:**
     - $g(n)$: Total cost from the start vertex to the current vertex, calculated as the sum of edge weights along the path.
     - $h(n)$: Estimated cost from the current vertex to the goal vertex using the selected distance metric.
     - $f(n) = g(n) + h(n)$
   - **Heuristic Modes:**
     - Mode 1: Manhattan distance — $h(n) = |x_1 - x_2| + |y_1 - y_2|$
     - Mode 2: Euclidean distance — $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
   - **Report Requirement:** Students must describe how $h(n)$ is calculated and provide a manual example run for this task.

3. **Maze Navigation with Obstacles (2.5 pts):** Given a maze represented as an $m \times n$ grid, where each cell is either passable (0) or blocked (1), implement the A* algorithm to find the shortest path from a start coordinate to a goal coordinate.

   - **Input:** Maze grid, start coordinate, goal coordinate.
   - **Output:** A linked list containing the shortest path represented as a sequence of directions using 8 possible moves:
     - Cardinal directions: Up, Down, Left, Right

  - Diagonal directions: Up-Left, Up-Right, Down-Left, Down-Right

  Each step should include the direction taken and the corresponding values of $f(n)$, $g(n)$, and $h(n)$. Each node in the linked list should be named according to the direction taken from that node (one of the eight directions). For example, if the first move from the start tile is Down, then the head node should be named "Down", with $g(n) = 1$ and $f(n)$ calculated using the Manhattan distance from the tile reached after moving Down to the goal tile.

- **Cost Function:**
  - $g(n)$: Total cost from the start node to the current cell. Horizontal and vertical moves cost 1 unit; diagonal moves cost 1.5 units.
  - $h(n)$: Estimated cost to the goal, calculated using Manhattan distance.
  - $f(n) = g(n) + h(n)$
- **Report Requirement:** Students must provide a manual example run for this task.

4. **Maze Navigation with Obstacles 2 (2.5 pts):** Given a maze represented as an $m \times n$ grid, where each cell is either passable (0) or blocked (1), rebuild the maze as a graph using a weight matrix. Each tile is treated as a vertex, with the mapping defined as: tile $(0, 0)$ is vertex 0, tile $(0, 1)$ is vertex 1, ..., tile (i, j) is vertex $i \cdot n + j$. Implement the A* algorithm on this graph to find the shortest path from a given start coordinate to a goal coordinate.

   - **Input:** Maze grid, start coordinate, goal coordinate, and an empty array to store the weight matrix.
   - **Output:** A linked list representing the path from start to goal. Each node in the list should include the values of $f(n)$, $g(n)$, and $h(n)$ at that vertex, along with the final computed weight matrix. Each node in the linked list should be named using its coordinate in the maze grid in the form (x, y).
   - **Cost Function:**
     - $g(n)$: Total cost from the start node to the current vertex. Horizontal and vertical moves cost 1 unit; diagonal moves cost 1.5 units.
     - $h(n)$: Estimated cost to the goal, calculated using Manhattan distance.
     - $f(n) = g(n) + h(n)$
   - **Report Requirement:** Students must describe the process of converting the maze to a weight matrix and provide a manual example run for this task.

2.2 Function Naming for Auto Testing

To facilitate automated testing, each programming task must be implemented as a function with the following standardized name and signature. The auto tester will use these function names to validate correctness.

- **Task 1 — Shortest Path Using A* with Adjacency Matrix:**
```
PathNode* findShortestPathMatrix(double adjMatrix[100][100],
int start, int goal);
```
- **Task 2 — Shortest Path in 2D Space with Two Heuristics:**
```
PathNode* findShortestPath2D(double adjMatrix[100][100], int coords[100][2],
int start, int goal, int mode);
```

- **Task 3 — Maze Navigation with Obstacles:**
  `PathNode* findPathInMaze(int maze[100][100], int m, int n, int startX, int startY, int goalX, int goalY);`
- **Task 4 — Maze Navigation with Obstacles 2:**
  `PathNode* findPathInMaze2(int maze[100][100], int m, int n, int startX, int startY, int goalX, int goalY, double weightMatrix[100][100]);`

Each function must return a pointer to the head of a linked list of `PathNode` objects, representing the path from start to goal. The list should include direction and cost values ($f(n)$, $g(n)$, $h(n)$) for each step.

2.3 Common Data Structure Requirement

For all four problems described in this project, students must define and use a common linked list data structure to represent the solution path. Each node in the linked list must contain the following fields:

- **Name:** a `std::string` representing the name or label of the node (e.g., vertex name, coordinate).
- $f(n)$**:** the total estimated cost from the start node to the goal through this node.
- $g(n)$**:** the actual cost from the start node to this node.
- $h(n)$**:** the heuristic estimate from this node to the goal.
- **Next:** a pointer to the next node in the path.

Example (C++ struct):

```
struct PathNode {
    std::string name;
    double f;
    double g;
    double h;
    PathNode* next;
};
```

This linked list should be used to store and output the final path found by the A* algorithm in each problem, preserving the order from the start node to the goal node. The following printPath function will be included in the LMS test case system for scoring. Make sure your struct works with it and **don't include this function in your LMS code runner submission**.

```
void printPath(PathNode* head) {
    cout << "Solution Path:\n";
    while (head != nullptr) {
        cout << "Node: " << head->name
                  << " | f: " << head->f
                  << " | g: " << head->g
                  << " | h: " << head->h << "\n";
        head = head->next;
    }
}
```

3 Q&A

3.1 Update List

- First Version

3.2 Q&A

If you have any questions about the assignment, you can use the Google form: https://forms.gle/5fy72BnCwQmNBoHc8.
Please check if your question is already answered in this section or the sheet https://docs.google.com/spreadsheets/d/11DlJRx4pOBdxHzWsvPwuSIOFuQZruE93QBfGIZ5tyTk/edit?usp=sharing
*If you ask an already answered question, you will receive a penalty point.

- None yet

4 Instructions and requirements

Students have to follow the instructions and comply with the requirements below. *Lecturers do not solve the cases arising because students do not follow the instructions or do not comply with the requirements.*

4.1 Instruction

- Deadline for submission: **November 30th, 2025**. Students have to answer each question clearly and coherently.
- Programming languages: C++

4.2 Submission

Students are required to complete 2 steps of submission to receive any assignment's score:

1. **Auto-testing code submission:** Submit your code to the appropriate auto-testcase runner on the LMS site. Each task mentioned above has a corresponding section in the submission interface. **(The submission will be available later)**
2. **File submission:** Place all the files specified in Section 2.1 into a folder named after your student ID (e.g., `2121323`). Then compress this folder into a `.rar` file with the same name (e.g., `2121323.rar`) before submitting it. The required files for submission are listed below:

   * The file submission is on the LMS video site and will be available on the last day of the deadline.
   * The penalty for each missing file or incorrect name, organizing is -1 point each.

4.3 Scoring

Your final score will be the harmonic mean of your **"code and report" score** and **"harmony questions" score** in favor of the **"code and report" score** using the following formula:

$$\min\left(A, \frac{3AB}{A+2B}\right)$$

where,
A is your code and report score.
B is the percentage of correct harmony questions in the final exam.
***In case your $A \geq 1$ and $B = 0$ you will get 1 point for submitting**

4.4 Cheating treatment

The assignment has to be done by individuals. A student will be considered as cheating if:

- There is an unusual similarity between the codes. In this case, ALL similar submissions are considered cheating. Therefore, the student must protect their work.
- They do not understand the works written by themselves. You can consult any source, but make sure that you know the meaning of what you have consulted and write your version.

Students will be judged according to the university's regulations if any part is found to be cheating.